



The purpose behind “week 4” in each course

- Welcome to the final week of the Kalman-Filter Boot Camp!
- It has been an intense experience, intended to strengthen your engineering skills to enable your success in the remaining specialization courses.
- I hope you have already found this tough process to be rewarding; I look forward to being able to pronounce “Victory!” with and for you at the end of this week.
- But first, my desire is that you find this week to be a special treat before continuing on to the second course, “Linear Kalman Filter Deep Dive.”
- The final week of each course focuses on an application of Kalman filters:
 - This course presents a state-estimation example;
 - The second course studies a target-tracking example;
 - The third course investigates a parameter-estimation example;
 - The final course looks at a navigation example.



Why state estimation?

- As you have learned, Kalman filters are “state estimators.”
- They combine a mathematical model of some system with input and output measurements to calculate the system’s most likely operating condition (state) at this moment, filtering random measurement noise.
- Sometimes, the estimate of the state is the final goal (e.g., target tracking); other times, finding a state estimate is an intermediate step in a more complex process.
 - For example, knowing a system’s state is necessary to solve many control-theory problems such as stabilizing a system using a state-feedback controller.
- In most practical cases, the physical state of the system cannot be determined by direct observation. Instead, indirect effects of the internal state are observed by way of the system outputs.
- The state-estimation example is the most direct application of KF theory, requiring the fewest additional assumptions and modifications to theory to implement.



Wait, we can implement a KF already?

- By now, you have learned the required background math concepts that will be needed to derive the KF steps.
- But, we’ll save those details until the next course, “Linear Kalman Filter Deep Dive.”
 - In my experience, it has been critical to understand the assumptions made when deriving the KF and the steps of the derivation, as I have found it to be necessary to re-derive the KF to use it for applications where the assumptions are not met.
 - But, when the assumptions are met exactly, we simply need to know what the KF steps are to be able to implement the KF and obtain state estimates.
- So, this week I will introduce the steps that we will derive in the second course, and I will show you how to implement a linear Kalman filter in Octave.
- You will execute this KF on a state-estimation problem by which you will learn how to interpret the output of the KF and get some intuition into how it works.



The KF prediction steps

Step 1a: State prediction time update

$$\hat{x}_k^- = A_{k-1}\hat{x}_{k-1}^+ + B_{k-1}u_{k-1}.$$

Step 1b: Prediction-error covariance time update

$$\Sigma_{\tilde{x},k}^- = A_{k-1}\Sigma_{\tilde{x},k-1}^+A_{k-1}^T + \Sigma_{\tilde{w}}.$$

Step 1c: Predict system output

$$\hat{z}_k = C_k\hat{x}_k^- + D_ku_k.$$

Prediction

- The KF has two principal steps (prediction and correction), each having three sub-steps. We look first at the prediction step.
- We define state prediction $\hat{x}_k^-$ to be our best guess of the state given all past information collected until time $k - 1$. $\Sigma_{\tilde{x},k}^-$ is the covariance (uncertainty) of $\hat{x}_k^-$.
- We define state estimate $\hat{x}_k^+$ to be our best guess of x_k given all information collected until time k . $\Sigma_{\tilde{x},k}^+$ is the covariance (uncertainty) of $\hat{x}_k^+$.
- The prediction steps predict the state, its error-covariance matrix, and the output.



The KF correction steps

Step 2a: Estimator gain matrix

$$L_k = \Sigma_{\tilde{x},k}^- C_k^T [C_k \Sigma_{\tilde{x},k}^- C_k^T + \Sigma_{\tilde{v}}]^{-1}.$$

Step 2b: State estimate measurement update

$$\hat{x}_k^+ = \hat{x}_k^- + L_k(z_k - \hat{z}_k).$$

Step 2c: Estimation-error covariance measurement update

$$\Sigma_{\tilde{x},k}^+ = (I - L_k C_k) \Sigma_{\tilde{x},k}^-.$$

Correction

- The feature of the Kalman filter that makes it optimal (in a sense we study next course) is how it computes its time-varying gain matrix.
- The estimator gain matrix L_k is used to blend between the state prediction and the new information in the measurement.
- The KF correction steps compute the estimator gain matrix, the state estimate, and its error-covariance matrix.
- The error-covariance matrix $\Sigma_{\tilde{x},k}^+$ is used to compute confidence bounds on $\hat{x}_k^+$.



The KF process

- After initializing $\hat{x}_0^+$ and $\Sigma_{\tilde{x},0}^+$ (somehow), we have enough information to jump into step 1 for $k = 1$; we then have enough information to jump into step 2 for $k = 1$.
- Then, we wait until the next timestep, and proceed to step 1 for $k = 2$ and then step 2 for $k = 2$ (and so forth).
- The output of the KF is $\hat{x}_k^+$ and $\Sigma_{\tilde{x},k}^+$ for every timestep.
- You will get a better intuitive feel for how this operates as we work through an example.
 - First, we will define the system for which we will investigate state estimation;
 - Then, you will learn how to write Octave code to implement a linear Kalman filter;
 - Finally, we will explore some example scenarios together.



Summary

- In the final week of each course in this specialization, we will study a specific application of Kalman filters.
- This week, our focus is on basic state estimation for a linear system, which might be part of a state-feedback control-system design.
- Even though you have not learned all the important assumptions made regarding a system and its environment during the Kalman-filter development, it's not too early to start experimenting with the KF steps to aid understanding.
- You learned the three substeps of both the prediction and correction major steps of the KF: six steps in total.
- After initialization, these steps are executed recursively every measurement interval.
- We are nearly ready to implement these steps in Octave code! But first, we will define the example problem that we will study this week.